

# Skill 3 · Base de Conversão LSP → Java

Versão: v1.7 · Interna · skill-03-base-conversao-lsp-java.md

Skill interna — **não** é fluxo de usuário. Aplique as regras globais do Router. Em conflito de assinatura, **revalide na Skill 2 / página oficial**.

**Fronteira:** Skill 2 = links/alias; **esta skill** = mecânica + catálogo + padrões de compilação + exemplos.

**Precedência:** docs oficiais Skill 2 → catálogo oficial abaixo → padrões `padrao_compilacao` (Eclipse) → anexos sanitizados → `validacao_manual` .

**Crescimento do catálogo confirmada** : só com doc oficial / Skill 2. Achados de compilação entram como `padrao_compilacao` , não como inventados.

## Como consultar (ordem obrigatória)

- A. Restrições + Regra de ouro + Invariantes ← nunca pular
- B. Workflow (6 passos)
- C. Tipos / sintaxe / VaPara / Funcao (se necessário)
- D. Catálogo – só a família do item LSP
- E. Acesso a dados (USU\_\* / R\* / ContextSession) se houver cursor/SQL
- F. Armadilhas Eclipse + checklist
- G. Exemplos – só o padrão análogo
- H. Saída tipada para a Skill 1

Não leia o catálogo inteiro de ponta a ponta. Localize a **família** no índice. Em conflito de assinatura, **revalide na Skill 2**.

## Índice rápido do arquivo

| Precisa de...                                  | Vá para                                         |
|------------------------------------------------|-------------------------------------------------|
| Proibições / minutos / Sem Situacao.getMinutos | Restrições absolutas                            |
| Modelo mental LSP→contexto                     | Regra de ouro                                   |
| Passo a passo                                  | Workflow                                        |
| Tipagem / Se→if / Logico / Escolha / VaPara    | Tipos e sintaxe                                 |
| Esqueleto Apuracao / Consistência / BH         | Esqueletos + tipos de regra                     |
| DatPro, HorSit, FPxMar, históricos...          | Catálogo → família                              |
| TipCon / MarcacaoRegra / overloads HorSit      | Catálogo + notas <code>padrao_compilacao</code> |
| Cursor USU_* / .sc / ContextSession            | Acesso a dados                                  |
| Erros Eclipse / anti-padrões                   | Armadilhas                                      |
| CalculaQtdMinutos / interjornada / TipCon SQL  | Exemplos sanitizados                            |
| Templates IEntity / .sc / DBCenter             | Acesso a dados                                  |

|                               |                       |
|-------------------------------|-----------------------|
| VaPara / operadores           | Tipos e sintaxe       |
| Goldens G-SIT / G-CUR / G-USU | Goldens ponta a ponta |
| InicioCalculo / AposGravar    | Outros pontos         |
| 🚫 SDK / ⚠️ pendente           | Itens pendentes (SDK) |
| Campos LSP conflitantes       | Tabela conflitantes   |

### Âncoras 80% (não leia o arquivo inteiro)

| Se a regra tem...                     | Vá direto para                                  |
|---------------------------------------|-------------------------------------------------|
| HorSit / situações / minutos          | Catálogo → Situações + <b>G-SIT</b>             |
| Marcações / FPxMar / MarcacaoAnterior | Catálogo → Marcações + Exemplos                 |
| TipCon / R034FUN                      | Campos conflitantes + TipCon via ContextSession |
| Cursor USU_* / interface / .sc        | Acesso a dados + <b>G-USU</b>                   |
| Cursor R* / CriarCursor               | Acesso a dados + <b>G-CUR</b> (API primeiro)    |
| VaPara / labels                       | Tipos e sintaxe → VaPara                        |
| INSERT/UPDATE/DELETE                  | DBCcenter (não ContextSession)                  |

### Fontes oficiais

| Fonte                              | URL                                                                                                                                                                                                                                                                                                                                                           | Uso                                    |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| Equivalência das funções de regras | <a href="https://documentacao.senior.com.br/gestao-de-pessoas-hcm/6.10.4/informacoes-adicionais/rotinas/gpo/integracao-controle-ponto-refeitorio/equivalencia-funcoes-regras.htm">https://documentacao.senior.com.br/gestao-de-pessoas-hcm/6.10.4/informacoes-adicionais/rotinas/gpo/integracao-controle-ponto-refeitorio/equivalencia-funcoes-regras.htm</a> | Mapa oficial                           |
| Índice das Funções HCM 6.10.4      | <a href="https://documentacao.senior.com.br/gestao-de-pessoas-hcm/6.10.4/customizacoes/funcoes.htm">https://documentacao.senior.com.br/gestao-de-pessoas-hcm/6.10.4/customizacoes/funcoes.htm</a>                                                                                                                                                             | Assinatura + <b>contexto</b> do método |

Catálogo abaixo: evidência confirmada (salvo nota). Exemplos: padrao\_anexo .

**Assinatura/contexto incompletos no catálogo** → abrir Índice das Funções (Skill 2) antes de gerar código.

### Quando usar / não usar

| Usar                                                      | Não usar                                              |
|-----------------------------------------------------------|-------------------------------------------------------|
| Converter/mapear variáveis e funções de apuração LSP→Java | Só mentoria; inventar método; só citar link (Skill 2) |

### Restrições absolutas

1. Preferir o catálogo; se faltar → Skill 2 → validacao\_manual (não inventar).

2. Horas em APIs de ponto = **minutos inteiros** ( `int` , não `double` solto).
3. SQL/cursor → API semântica do catálogo **antes** de `EntitySession/ContextSession` — exceto tabela `USU_* custom` sem API (aí `ICursor+ .sc` ).
4. **Proibido** `getSituacao(...).getMinutos()` / `setMinutos(...)` — use `getHorSit` / `setHorSit` / `zeraHorasSituacao` .
5. Sanitize nomes de cliente em exemplos anexados; package do projeto é **parâmetro** (ex. sanitizado `gestaopontoCustom` ) — nunca vaziar cliente.
6. Sem `Mensagem()` /popup em apuração — preferir `mensagemLog` .
7. Confirmar se o método existe no **contexto** da regra (apuração ≠ consistência ≠ BH ≠ geral).
8. `execute()` **sem** `throws Exception` ; auxiliares `private` no **nível da classe** (nunca aninhados).
9. `ContextSession/DBCcenter` da plataforma ≠ **Senior SQL 2** (SQL 2 continua proibido).

## Regra de ouro

LSP e Java diferem no **modelo de execução**: variáveis/funções globais LSP → **métodos no objeto de contexto**.

Quem só troca `Inicio/Fim` por `{ }` produz código que não compila.

## Workflow (6 passos)

1. Identificar contexto (apuração / consistência / bloqueio / fechamento BH / geral).
2. Inventariar construções LSP.
3. Mapear no catálogo (nome **e** ordem de parâmetros).
4. Traduzir mecânica (getters/setters, arrays→métodos, cursor→ `List` , `End` →retorno).
5. Traduzir sintaxe ( `Se` → `if` , `=` comparação→ `==` / `.isEqual()` , etc.).
6. Revisar armadilhas.

## Invariantes

1. Toda variável/função de contexto LSP → chamada de método.
2. Não inventar assinatura.
3. Conferir ordem de parâmetros.
4. Horas = minutos inteiros.
5. Tipagem forte Java.
6. Sem popup em apuração.
7. Arrays indexados → métodos/coleções (não `getApuDiu(1)` inventado).
8. Método deve existir no **contexto** da regra.

## Tipos e sintaxe

| LSP                 | Java                                                                                              |
|---------------------|---------------------------------------------------------------------------------------------------|
| Numero inteiro      | <code>int</code> / <code>long</code>                                                              |
| Numero decimal      | <code>double</code>                                                                               |
| Alfa                | <code>String</code>                                                                               |
| Logico / lógico 0/1 | <code>boolean</code> ( <code>0</code> → <code>false</code> , <code>1</code> → <code>true</code> ) |

|                                    |                                                                              |
|------------------------------------|------------------------------------------------------------------------------|
| Data                               | LocalDate (ou padrão do SDK — não misturar Joda + java.time sem necessidade) |
| Hora (minutos)                     | int minutos                                                                  |
| marcação                           | MarcacaoRegra (padrao_compilacao; preferir a Marcacao genérica)              |
| Se / Senao / Enquanto / Inicio/Fim | if / else / while / { }                                                      |
| Escolha / Caso / OutroCaso         | switch (sem fall-through accidental) <b>ou</b> if/else encadeado             |
| Vapara / labels                    | early return; break/continue; ou flag + do { } while(false)                  |
| Funcao sem end                     | private void no nível da classe                                              |
| Funcao com 1 end                   | retorno tipado do método                                                     |
| Funcao com vários end              | int[] / objeto (não closures aninhadas)                                      |
| Numero[n] array                    | int[n+1] ou double[n+1] (LSP 1-based → Java 0-based)                         |
| Tabela                             | List<record> / classe interna                                                |
| @ ... @                            | //                                                                           |
| = comparação                       | == / .isEqual() / .equals()                                                  |
| <>                                 | !=                                                                           |
| decimal ,                          | .                                                                            |
| e / ou / nao                       | && / `                                                                       |
| RestoDivisao(a,b)                  | a % b                                                                        |
| Mensagem(...) em regra             | exceção de domínio (BusinessException / RegraApuracaoException) — não popup  |

## Escolha / Caso ( padrao\_compilacao )

```
Escolha (n0pcao)
  Caso 1
    @ ...
  Caso 2
    @ ...
  OutroCaso
    @ ...
FimEscolha;
```

```
switch (n0pcao) {
  case 1:
    // ...
    break;
```

```

    case 2:
        // ...
        break;
    default:
        // OutroCaso
        break;
}

```

Alfa/ String : preferir `if/else` com `.equals()` (ou `switch` em Java 17+ com cuidado). **Não** copiar fall-through do `switch` sem `break` a menos que o LSP caia propositalmente no próximo `Caso` .

## VaPara — 3 padrões ( `padrao_compilacao` )

### 1. Early return

```

Se (condicao) { VaPara FimRegra; }
@ lógica
FimRegra:

```

```

if (condicao) return;
// lógica

```

### 2. Flag + do-while false (bloco “pulado” com label)

```

boolean fezProrrogacao = false;
do {
    // bloco principal
    fezProrrogacao = true;
} while (false);
if (!fezProrrogacao) { /* bloco do label */ }

```

### 3. Break de loop

```

while (condicao) {
    if (encerrar) break;
}

```

## Para (0-based em Java)

```

for (int i = 0; i < n; i++) { /* LSP Para costuma 1..n */ }

```

## Instruções

1. Seguir “Como consultar” (A→H)
2. Identificar contexto
3. Buscar item só na família do Catálogo

4. Assinatura/contexto duvidosos → Índice das Funções (Skill 2)
5. Ausente → validacao\_manual

## Tipos de regra e esqueletos ( padrao\_compilacao )

| Tipo LSP      | Classe base                                | Contexto                        |       |
|---------------|--------------------------------------------|---------------------------------|-------|
| Apuração      | custom.senior.apuracao.Apuracao            | getContextoApuracao()           | Regra |
| Consistência  | custom.senior.apuracao.ConsistenciaAcertos | getContextoConsistenciaAcerto() | Regra |
| Manutenção BH | custom.senior.bancohoras.RegraManutencaoBH | getContextoManutencaoBH()       | Regra |
| Fechamento BH | custom.senior.bancohoras.RegraFechamentoBH | getContextoFechamentoBH()       | Regra |

Package: o informado pelo usuário/projeto. Exemplo sanitizado: `gestaopontoCustom` (não `gestaoDoPontoCustom` ).

### Apuração

```
@Rule(description = "Regra de Apuracao")
public class RegraApuracao extends Apuracao {
    @Override
    public void execute() { // SEM throws Exception
        ContextoGeralRH ctxGeral = getContainer().getContextoGeral();
        ContextoApuracao ctx = getContainer().getContextoApuracao();
    }

    private void auxiliar(ContextoApuracao ctx, int param) { /* nível da classe */ }
}
```

### Consistência de Acertos

```
@Rule(description = "Consistencia de Acertos")
public class RegraConsistencia extends ConsistenciaAcertos {
    @Override
    public void execute() {
        ContextoConsistenciaAcerto ctx =
getContainer().getContextoConsistenciaAcerto();
        // extras: getHorSitAnterior, getDataInicial/Final, getHorSitAnteriorFaixa
    }
}
```

### Fechamento BH

```
@Rule(description = "Fechamento BH")
public class RegraFechamentoBH extends custom.senior.bancohoras.RegraFechamentoBH {
```

```

@Override
public void execute() {
    ContextoFechamentoBH ctx = getContainer().getContextoFechamentoBH();
}
}

```

Imports frequentes ( padrao\_compilacao ): java.time.LocalDate / LocalDateTime , com.senior.rule.Rule , com.senior.dataset.ICursor , MappedParamProvider , Colaborador , MarcacaoRegra , TipoIntervalo , TipoHoraExtra , ContextSession , DBCenter , IResultSet , com.senior.dataset.IEntity ( + @Entity / @Field ), readonly IR030EMP / IR060DSI /...

## Catálogo de equivalência (oficial Senior 6.10.4)

Fonte: [Equivalência das funções de regras](#).

Métodos tipicamente em contextoApuracao / container — **confirmar contexto e assinatura** no Índice das Funções.

### Famílias (âncora de busca)

| Família                    | Itens típicos LSP                       |
|----------------------------|-----------------------------------------|
| Data / sistema             | DatPro, HorSis, DiaSem, DatIni...       |
| Situações                  | HorSit[], SitAnt[], TotSit[], MotSit    |
| Horas apuradas/previstas   | ApuDiu[], ApuNot[], TraDiu, PrvTra[]... |
| Marcações                  | FPxMar, FLeMar, DatMar, QtdMar...       |
| Escala / horário / feriado | EscAtu, CodHor, FerFil...               |
| Históricos / usuário       | CarEmp, CodSinEmp, CodUsu...            |
| Compensação / BH / log     | TemCmp[], RetBHRDat, MensagemLog...     |

### Data / sistema

| LSP (Controle de Ponto)               | Java (Gestão do Ponto)               |
|---------------------------------------|--------------------------------------|
| DatPro (implícito via Ano/Dia/MesPro) | getData()                            |
| AnoPro                                | getAnoData(Date data) / getData()    |
| MesPro                                | getMesData(Date data)                |
| DiaPro                                | getDiaData(Date data)                |
| AnoSis / MesSis / DiaSis              | getAnoData / getMesData / getDiaData |
| HorSis / ExtSis                       | Nativo em Java                       |
| DatIni / Datfim                       | getDataInicial() / getDataFinal()    |

|                                                  |                                                                                                                                  |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| DiaSem, DiaDom...DiaSab                          | Preferir <code>getDiaSem(Date)</code> ; alt. <code>getData().getDayOfWeek().getValue()</code> ( <code>padrao_compilacao</code> ) |
| MesAtu                                           | <code>getMesData(Date data)</code>                                                                                               |
| RetornaAnoData / RetornaDiaData / RetornaMesData | <code>getAnoData / getDiaData / getMesData</code>                                                                                |

## Situações / HorSit

| LSP                                    | Java                                                                                                                                                                                                                                      |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HorSit[]                               | <code>getHorSit(int) / getHorSit(LocalDate, int), setHorSit, somaHorasSituacao, zeraHorasSituacao, zeraHorasSituacaoFaixa, getHorSitFaixa</code> — overloads <code>padrao_compilacao: getHorSit(int[]), zeraHorasSituacao(varargs)</code> |
| SitAnt[]                               | <code>getHorSitAnterior(int) / getHorSitAnteriorFaixa</code> (consistência)                                                                                                                                                               |
| TotSit[] / BuscaTotalizadoresSituacoes | <code>getTotalSituacoes(int codigoTotalizador, date data)</code> (e overload com intervalo)                                                                                                                                               |
| MotSit                                 | <code>getMotivoAcerto(int situacao)</code>                                                                                                                                                                                                |

## Apuração diurna/noturna / trabalhadas / previstas

| LSP                                                                                                      | Java                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ApuDiu[] / ApuNot[]                                                                                      | <code>getHorasSeparadas(TipoIntervalo TipoHoraExtra) → ISeparacaoHoras (getHorasDiurnas/Noturnas/TotalHoras);</code> OU <code>getHoras(MarcacaoRegra, ...)</code> |
| HrtRaD[] / HrtRan[] / TraDiu / TraNot                                                                    | <code>getHorasTrabalhadas(int parte)</code>                                                                                                                       |
| MinPvd[] / MinPvn[] / PrvTra[] / PrvTrd / PrvTrn / VprVho / VprVin / VprVtr                              | <code>getTotalMinutosPrevisto(...)</code> / <code>getHorasPrevistas(...)</code> / <code>getTotalMinutosPrevistoProrrogado(...)</code>                             |
| FatRed                                                                                                   | <code>getHorarioPrevistoColaborador(...)</code>                                                                                                                   |
| ExtrasIntervalo                                                                                          | <code>getExtrasIntervalo / getExtrasIntervaloAnterior / getExtrasIntervaloPosterior</code>                                                                        |
| MinExD[] / MinExn[] / MsaInd[] / MsaInn[] / IniExt / FimExt / PerExt / PriExt / QtdExt / TipExt / DiaExt | <code>getIntervalosCalculados()</code> / <code>getIntervaloCalculado(int indice)</code>                                                                           |

## Marcações

| LSP                   | Java                                                                                                                                                      |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| FPxMar / FLeMar / ... | <code>List&lt;MarcacaoRegra&gt; getMarcacoesRealizadas(boolean)</code> — <code>getData()</code> → <code>.toLocalDate();</code> índice Java <b>0-based</b> |



|                             |                                                                                                                       |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------|
| QtdMar / TotMar             | Preferir getQtdMarcacoesRealizadas(boolean); alt. .size() (padrao_compilacao)                                         |
| DulMar / HulMar             | getMarcacaoAnterior() → tipo MarcacaoAnterior: <b>só</b> diferencaMinutos(data, hora) — sem getHora/getData           |
| MinJor                      | Preferir getHorasInterjornadaRealizada(); alt. diferencaMinutos se só MarcacaoAnterior                                |
| MinMJo                      | getHorasInterjornadaPrevista()                                                                                        |
| TipCon                      | <b>Não</b> existe Colaborador.getTipCon() — SQL SELECT TIPCON FROM R034FUN ... via ContextSession (padrao_compilacao) |
| ApuDiu/ApuNot<br>tipados    | Também getHorasSeparadas(TipoIntervalo TipoHoraExtra) → ISeparacaoHoras                                               |
| log / pendência /<br>alerta | mensagemLog, setGerarPendencia, criarAlerta (try/catch)                                                               |

### Escala / horário / filial / feriado

| LSP                                      | Java                                                                                   |
|------------------------------------------|----------------------------------------------------------------------------------------|
| EscAtu                                   | getEscala()                                                                            |
| EscEmp / RetEscEmp                       | getHistoricoEscala()                                                                   |
| EscTrf                                   | getEscalaHistorico()                                                                   |
| TemTes                                   | getTrocaEscala(Date data)                                                              |
| TemThr                                   | getTrocaHorario(Date data)                                                             |
| RetornaEscala                            | getEscalaPrevistaColaborador(...)                                                      |
| CodHor / RetornaHorarioApurado           | getHorario() / getHorario().getCodigo() — especiais 9996–9999 (folga/feriado/comp/DSR) |
| HorEsc / HorTrf                          | getHorarioEscala()                                                                     |
| HorDFe                                   | getHorarioOriginalEscala()                                                             |
| HorFol                                   | getCodigoHorarioFolga() / getHorarioFolga()                                            |
| HorPfo                                   | getHorarioProjecaoFolga()                                                              |
| RetornaHorario /<br>RetornaBatidaHorario | getHorarioPrevistoColaborador(...)                                                     |
| NumPer                                   | getNumeroPeriodos(int codHor)                                                          |
| NumInt                                   | getNumeroIntervalos()                                                                  |
| NinRef                                   | getNumeroIntervaloRefeicao()                                                           |
| TurInt                                   | getTurmaIntervalo()                                                                    |

|                                            |                              |
|--------------------------------------------|------------------------------|
| RetMinRefHTr                               | getMinutosRefeicaoPrevisto() |
| FilEmp / RetFilEmp / DatAltFil / EmpAltFil | getHistoricoFilial()         |
| FerFil                                     | getFeriadoFilial(Date data)  |
| VerDatFer                                  | getFeriado(Date data)        |

## Históricos colaborador / usuário

| LSP                                          | Java                                                    |
|----------------------------------------------|---------------------------------------------------------|
| CarEmp / EstCarEmp / RetCarEmp               | getHistoricoCargo()                                     |
| CcuEmp / DatAltCcu / RetCcuEmp               | getHistoricoCentrodeCusto()                             |
| CodSinEmp / RetSinEmp                        | getHistoricoSindicato()                                 |
| CodVinEmp / RetVinEmp                        | getHistoricoVinculo()                                   |
| LocEmp / RetLocEmp                           | getHistoricoLocal(...)                                  |
| CodAfs / IniAfs / FimAfs / TemAfs / QtdAfs[] | getHistoricosAfastamento()                              |
| BusCraTit                                    | getHistoricosCracha() / getHistoricosCrachaProvisorio() |
| RetApuPon                                    | getHistoricoApuracao(colaborador, data)                 |
| CodUsu                                       | getUsuarioAtivo()                                       |
| NomUsu                                       | getUsuario(long codigoUsuario)                          |
| RetornaDesGrupo / RetornaQtdGrupos           | getGrupos(long codigoUsuario)                           |
| AssociaUsuColab                              | associarUsuarioColaborador(...)                         |
| RetColabPorCodUsu                            | getUsuarioColaborador(...)                              |

## Compensação / banco de horas / cálculo

| LSP                                                                                   | Java                                                                          |
|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| TemCmp[] / DtICmp[] / DtFCmp[] / PerCmp[] / QtdCmp[] / SitCmD[] / SitCmN[] / TipCmp[] | getCompensacoes() / getCompensacoes(LocalDate data)                           |
| LimBa1 / LimBa2                                                                       | getEscala()                                                                   |
| RetBHRDat                                                                             | getSaldoBanco(int banco, int empresa, int tipo, int cadastro, LocalDate data) |
| TipCal                                                                                | getTipoCalculo()                                                              |
| MensagemLog                                                                           | mensagemLog(String mensagem)                                                  |
| VerificaAbrangenciaNumero                                                             | getAbrangencia(string abrangencia, int numero)                                |

|                                           |                                                     |
|-------------------------------------------|-----------------------------------------------------|
| RetornaCodLoc / RetornaNumLoc / RetNivLoc | getCodigoLocal / getNumeroLocal / getQtdNiveisLocal |
|-------------------------------------------|-----------------------------------------------------|

Definição de situações (padrão complementar)

Cursor R014SIN / R030EMP para CodDsi → preferir API ( padrao\_anexo até confirmar na doc):

```
int codDsi = contextoApuracao.getDefinicaoSituacoes().getCodigo();
```

Campos LSP conflitantes ( padrao\_compilacao )

| LSP              | Erro comum                   | Correto                                       |
|------------------|------------------------------|-----------------------------------------------|
| TipCon           | col.getTipCon()              | SQL R034FUN.TIPCON                            |
| nMar / marcações | índice 1-based / tipo errado | List<MarcacaoRegra> 0-based + .toLocaleDate() |
| MarcacaoAnterior | getHora() / getData()        | só diferencaMinutos                           |
| HorSit[n]        | tratar como array Java       | get/setHorSit(codSit, minutos)                |
| SitAnt           | getter inventado             | getHorSitAnterior no contexto de consistência |
| QtdMar           | inventar campo               | API oficial ou .size()                        |

Domínio HCM — constantes úteis

| Valor            | Uso                                                         |
|------------------|-------------------------------------------------------------|
| CodHor 9996–9999 | Folga / feriado / compensado / DSR (<9996 = horário normal) |
| 1320 / 300       | Janela noturna 22h–05h (minutos)                            |
| 660 / 360        | Limites CLT interjornada / sem intervalo                    |
| 1440             | Virada de dia em CalculaQtdMinutos                          |

Acesso a dados (após API-first)

```
Tabela USU_* custom?
SIM → IEntity + .sc + ICursor (EntitySession)
NÃO (R* nativa / USU_* em tabela nativa) → ContextSession SELECT ou DBCenter DML
Readonly em com.senior.rh.entities.readonly.*? → importar, não recriar
```

Fatal: .sc apontando para Rxxxxx → "Tabela R034FUN inválida. É permitido mapear somente tabelas de usuário".

ICursor — uma linha

```

MappedParamProvider params = new MappedParamProvider();
params.setParam("vNumEmp", vNumEmp);
ICursor<IMinhaTabela> cur =
getContainer().getEntitySession().newCursor(IMinhaTabela.class);
cur.addFilter("USU_NumEmp = :vNumEmp", params);
cur.open();
try {
    if (cur.first()) { IMinhaTabela row = cur.read(); }
} finally { cur.close(); }

```

## ICursor — múltiplas linhas + ordem

```

String[] campos = new String[]{"USU_DatAlt"};
OrderDirection[] ordem = new OrderDirection[]{OrderDirection.DESC};
cur.setOrder(campos, ordem);
cur.open();
try {
    while (cur.next()) {
        IMinhaTabela row = cur.read();
    }
} finally { cur.close(); }

```

## Três formas de EntitySession

```

// 1) preferencial no execute()
getContainer().getEntitySession().newCursor(T.class);

// 2) auxiliar fora do execute()
IEntitySession es = EntitySessionProvider.getSession();
es.newCursor(T.class);

// 3) chave exata (CursorUtil)
ICursor<T> cur = CursorUtil.getCursor(T.class);
T buffer = cur.newBuffer();
buffer.setNumEmp(col.getNumEmp());
cur.open();
try {
    if (cur.search(buffer, SearchMode.EXACT_MATCH)) { cur.read(buffer); }
} finally { cur.close(); }

```

## ContextSession (SELECT) vs DBCenter (DML)

|             | ContextSession                | DBCenter                 |
|-------------|-------------------------------|--------------------------|
| Uso         | SELECT                        | INSERT/UPDATE/DELETE     |
| Close       | <b>Não</b> fechar (container) | <b>Fechar</b> no finally |
| Placeholder | ?                             | ?                        |

|                   |                                      |                                  |
|-------------------|--------------------------------------|----------------------------------|
| Índice IResultSet | <b>base-0</b> (getInt(0) = 1º campo) | —                                |
| databaseId        | —                                    | ex. "vetorh" (confirmar projeto) |

### DBCenter INSERT (sanitizado)

```

IDBCenter database = DBCenter.getInstance("vetorh"); // confirmar databaseId do
projeto
ISession session = null;
try {
    session = database.newSession();
    Date dataConv =
Date.from(dataPro.atStartOfDay(ZoneId.systemDefault()).toInstant());
    String dataStr = new SimpleDateFormat("dd/MM/yyyy").format(dataConv);
    session.executeUpdate(
        "INSERT INTO USU_TExemplo (USU_NumEmp, USU_Dat, USU_CodSit) VALUES (?, ?,
?),",
        numEmp, dataStr, codSit);
} catch (Exception ex) {
    ctx.mensagemLog("Erro DML: " + ex.getMessage());
} finally {
    if (session != null) session.close();
}

```

### Template IEntity (só USU\_\*)

```

package gestaopontoCustom; // package do projeto – sanitizar

import java.time.LocalDate;
import com.senior.dataset.IEntity;
import com.senior.dataset.annotation.Entity;
import com.senior.dataset.annotation.Field;

@Entity
public interface IMinhaTabela extends IEntity {

    @Field(description = "Empresa")
    int getUSU_CodEmp();
    void setUSU_CodEmp(int USU_CodEmp);
    boolean isUSU_CodEmpNull();
    void setUSU_CodEmpNull();

    @Field(description = "Data")
    LocalDate getUSU_DatEmi();
    void setUSU_DatEmi(LocalDate USU_DatEmi);
    boolean isUSU_DatEmiNull();
    void setUSU_DatEmiNull();

    @Field(description = "Usuário – long")
    long getUSU_CodUsu();
}

```

```

    void setUSU_CodUsu(long USU_CodUsu);
    boolean isUSU_CodUsuNull();
    void setUSU_CodUsuNull();
}

```

Null: `int v = entity.isUSU_CodEmpNull() ? 0 : entity.getUSU_CodEmp();`

IDs de usuário/consultor → **long**, não **int**.

Import correto: `com.senior.dataset.IEntity` — **nunca** `com.senior.g5...IEntity`.

**Norma capitalização** `USU_*`: getters/setters/null **espelham o nome do campo** exatamente como no banco/ `.sc` (ex.: campo `USU_CodEmp` → `getUSU_CodEmp` / `setUSU_CodEmp`). Não “javaizar” para `getUsuCodEmp`. Divergência só com evidência do schema do projeto.

## Template `.sc` (JSON puro)

```

{
  "datasets": [
    {
      "id": "ScMinhaTabela",
      "entity": "gestaopontoCustom.IMinhaTabela",
      "storage": {
        "databaseId": "vetorh",
        "query": "from USU_MINHATAB USU_MINHATAB",
        "fieldMapping": [
          { "field": "USU_CodEmp", "column": "USU_MINHATAB.USU_CodEmp" },
          { "field": "USU_DatEmi", "column": "USU_MINHATAB.USU_DatEmi" },
          { "field": "USU_CodUsu", "column": "USU_MINHATAB.USU_CodUsu" }
        ],
        "type": "DB"
      }
    }
  ]
}

```

Regras: `id` = **nome do arquivo sem extensão**; começa com `{`; sem BOM/comentários; só `USU_*` no `query`.

## Readonly do framework (importar)

`IR030EMP`, `IR060DSI`, `IR004HOR`, `IR010SIT`, `IR010TOB`, `IR014SIN`, `IR070ACC`, ...

## Getters encadeados / `ISeparacaoHoras` ( `padrao_compilacao` )

```

int codSin = ctx.getHistoricoSindicato().getCodSin();
int tabOrg = ctx.getHistoricoLocal().getTabOrg();
int numLoc = ctx.getHistoricoLocal().getNumLoc();
HistoricoVinculo hv = ctx.getHistoricoVinculo();
if (hv != null) { /* usar hv */ }

ISeparacaoHoras sep = ctx.getHorasSeparadas(TipoIntervalo.REFEICA0);

```

```
int diu = sep.getHorasDiurnas();
int not = sep.getHorasNoturnas();
int tot = sep.getTotalHoras();
```

## Armadilhas práticas

| Armadilha / erro Eclipse               | Correção                                    |
|----------------------------------------|---------------------------------------------|
| Só trocar sintaxe sem mapear variáveis | Inventário + getters/setters primeiro       |
| Copiar ordem de parâmetros LSP         | Confirmar no Índice das Funções             |
| getSituacao().get/setMinutos           | getHorSit / setHorSit / zeraHorasSituacao   |
| Cursor SQL por reflexo                 | Buscar família no catálogo; USU_* → ICursor |
| getHorSit(variavelDeMinutos)           | 1º arg = <b>código da situação</b>          |
| Inventar getApuDiu(1)                  | getHoras / getHorasSeparadas                |
| execute() throws Exception             | Remover throws (IRulePoint)                 |
| Método aninhado / código solto         | Auxiliares no nível da classe               |
| col.getTipCon()                        | SQL R034FUN.TIPCON                          |
| MarcacaoAnterior.getHora/getData       | diferencaMinutos                            |
| LocalDateTime → LocalDate              | .toLocalDate() em MarcacaoRegra.getData()   |
| .sc em tabela R*                       | Remover .sc; ContextSession                 |
| .sc não-JSON / BOM                     | Reescrever começando com {                  |
| import com.senior.g5...IEntity         | com.senior.dataset.IEntity                  |
| Package gestaoDoPontoCustom            | Package do projeto (ex. gestaopontoCustom)  |
| IRResultSet base-1                     | base-0                                      |
| Filename ≠ classe pública              | Renomear arquivo (Windows: temp rename)     |
| Inventar método                        | validacao_manual                            |
| Misturar java.time e Joda              | Preferir java.time / SDK                    |

## Mensagens Eclipse literais (referência)

| Mensagem                                                     | Correção                             |
|--------------------------------------------------------------|--------------------------------------|
| The import com.senior.g5 cannot be resolved                  | import<br>com.senior.dataset.IEntity |
| execute() throws Exception not declared by IRulePoint        | Remover throws Exception             |
| The method getTipCon() is undefined for the type Colaborador | SQL R034FUN.TIPCON                   |

|                                                                     |                               |
|---------------------------------------------------------------------|-------------------------------|
| The method getHora() is undefined for the type MarcacaoAnterior     | diferencaMinutos              |
| Type mismatch: cannot convert LocalDateTime to LocalDate            | .toLocalDate()                |
| Expected BEGIN_OBJECT but was STRING                                | .sc JSON puro começando com { |
| Tabela Rxxx inválida. É permitido mapear somente tabelas de usuário | Sem .sc em R*                 |
| The public type X must be defined in its own file                   | Filename = nome da classe     |

**Windows (case):** Refactor → nome temporário → nome correto (ou `ren` em duas etapas no CMD).

## Exemplos sanitizados (use só o análogo)

### HorSis + setHorSit

```
Se (HorSis > 870) Inicio
  HorSit = 120;   @ situação 100 @
Fim;
```

```
java.time.LocalTime agora = java.time.LocalTime.now();
int minutos = agora.getHour() * 60 + agora.getMinute();
if (minutos > 870) {
    contextoApuracao.setHorSit(100, 120);
}
```

### HorSit — errado vs certo

```
// ERRADO
contexto.getSituacao(xNorAnt).setMinutos(0);
// CERTO
contexto.zeraHorasSituacao(xNorAnt);
int v = contexto.getHorSit(xExDDsr); // minutos int
contexto.setHorSit(xExDDsr, v + vNorDes);
```

### Marcações FLeMar/FPxMar → List

```
for (MarcacaoRegra m : contextoApuracao.getMarcacoesRealizadas(false)) {
    LocalDate d = m.getData().toLocalDate();
    if (d.isAfter(contextoApuracao.getData())) { /* ... */ }
}
```

### End → retorno ( RetBHRDat )

```
// ordem Java ≠ LSP – confirmar no índice
int saldo = contextoApuracao.getSaldoBanco(codbhr, numemp, tipcol, numcad,
```



```
new LocalDate(2014, 9, 24));
```

### Múltiplos End → objeto

```
private ResultadoDiurnoNoturno calculaDiurnoNoturno(...) {  
    return new ResultadoDiurnoNoturno(minutosDiurnos, minutosNoturnos);  
}
```

### Troca de situação

```
private int trocarSituacao(ContextoApuracao ctx, int origem, int destino, int minutosPendentes) {  
    int minutosOrigem = ctx.getHorSit(origem);  
    if (minutosOrigem >= minutosPendentes && minutosOrigem > 0 && minutosPendentes > 0) {  
        ctx.setHorSit(destino, minutosPendentes);  
        ctx.setHorSit(origem, minutosOrigem - minutosPendentes);  
        return 0;  
    }  
    if (minutosOrigem < minutosPendentes && minutosOrigem > 0 && minutosPendentes > 0) {  
        ctx.setHorSit(destino, minutosOrigem);  
        ctx.setHorSit(origem, 0);  
        return minutosPendentes - minutosOrigem;  
    }  
    return minutosPendentes;  
}
```

### CalculaQtdMinutos (virada de dia)

```
private int calculaQtdMinutos(LocalDate d1, int horIni, LocalDate d2, int horFim) {  
    if (d1.isBefore(d2)) horFim = horFim + 1440;  
    return horFim - horIni;  
}
```

### TipCon via ContextSession

```
int nTipCon = 0;  
try {  
    ResultSet rs = ContextSession.getSession().executeQuery(  
        "SELECT TIPCON FROM R034FUN WHERE NUMEMP = ? AND TIPCOL = ? AND NUMCAD = ?",  
        numEmp, tipCol, numCad);  
    if (rs.next()) nTipCon = rs.getInt(0);  
} catch (Exception ex) {  
    ctx.mensagemLog("Erro TipCon: " + ex.getMessage());  
}
```

## Interjornada (MarcacaoAnterior)

```
if (ctx.getMarcacaoAnterior() != null) {
    int n = ctx.getMarcacaoAnterior().diferencaMinutos(datMar, horaMar);
    if (n < 0) n = -n;
    // preferir getHorasInterjornada* quando disponível no contexto
}
```

## EntitySession / ICursor (USU\_\* sem API)

```
ICursor<IEntidadeCustom> cursor = getContainer().getEntitySession()
    .newCursor(IEntidadeCustom.class);
try {
    cursor.addFilter("USU_Campo = :v", new MappedParamProvider("v", valor));
    cursor.open();
    if (cursor.first()) { /* read */ }
} finally {
    cursor.close();
}
```

Justificar ausência de API semântica; emitir interface+ `.sc` só para `USU_*` .

## DiferencaMarcacaoDiurnoNoturno (sanitizado)

Preferir `getHorasSeparadas` / APIs oficiais quando cobrirem o caso. Se a regra LSP calcular manualmente diurno/noturno na janela 22h–05h ( `1320` / `300` ):

```
/** Retorna [diurno, noturno] em minutos. Janela noturna padrão: 1320→1440 e 0→300.
 */
private int[] diferencaMarcacaoDiurnoNoturno(int nMar1, LocalDate d1, int nMar2,
LocalDate d2,
int iniNot, int fimNot) {
    int total = calculaQtdMinutos(d1, nMar1, d2, nMar2);
    if (total < 0) total = -total;
    int noturno = 0;
    int cursor = nMar1;
    LocalDate dia = d1;
    int restante = total;
    while (restante > 0) {
        int limiteNot = (cursor >= iniNot) ? 1440 : (cursor < fimNot ? fimNot :
iniNot);
        int trecho;
        if (cursor >= iniNot || cursor < fimNot) {
            trecho = Math.min(restante, (cursor >= iniNot ? 1440 - cursor : fimNot -
cursor));
            if (trecho <= 0) trecho = Math.min(restante, 1);
            noturno += trecho;
        } else {
            trecho = Math.min(restante, iniNot - cursor);
        }
    }
}
```

```

    }
    cursor += trecho;
    restante -= trecho;
    if (cursor >= 1440) {
        cursor -= 1440;
        dia = dia.plusDays(1);
    }
}
return new int[]{total - noturno, noturno};
}

```

**Proibido** publicar `return new int[]{0,0}` ou `// preencher` — isso é **CHK-STUB FAIL**.

### VerificaInterjornada (MarcacaoAnterior)

```

private void verificaInterjornada(ContextoApuracao ctx, LocalDate datMar, int
horaMar, int sitInt) {
    if (ctx.getMarcacaoAnterior() == null) return;
    // Preferir getHorasInterjornadaRealizada/Prevista quando existirem no contexto
    int n = ctx.getMarcacaoAnterior().diferencaMinutos(datMar, horaMar);
    if (n < 0) n = -n;
    if (n > 0 && n < 660) {
        ctx.setHorSit(sitInt, 660 - n);
    }
}

```

### MarcacoesInvalidas (padrão)

```

/** Pares entrada/saída: lista vazia ou ímpar = inválida; data/hora de saída antes
da entrada = inválida. */
private boolean marcacoesInvalidas(List<MarcacaoRegra> mars) {
    if (mars == null || mars.isEmpty()) return true;
    if (mars.size() % 2 != 0) return true;
    for (int i = 0; i + 1 < mars.size(); i += 2) {
        // getData() tipicamente LocalDateTime – comparar o instante completo
        if (mars.get(i + 1).getData().isBefore(mars.get(i).getData())) {
            return true;
        }
    }
    return false;
}

```

Regras de negócio extras do cliente (tolerâncias, tipos de marcação) → `validacao_manual` / anexo — **sem** deixar o método vazio.

### Outros pontos de customização (referência, não fluxo de menu)

| Tipo | Base | APIs típicas |
|------|------|--------------|
|------|------|--------------|

|                          |                          |                                                                         |
|--------------------------|--------------------------|-------------------------------------------------------------------------|
| InicioCalculoColaborador | InicioCalculoColaborador | getMarcacoes, getTrocaHorario, incluirTrocaHorario, Periodo.getInstance |
| AposGravarApuracao       | AposGravarApuracao       | isTelaAcerto, getDiasCalculados, isUltimoColaborador                    |
| Manutenção BH            | RegraManutencaoBH        | getLancamentos() → ILancamentoBH                                        |
| Fechamento BH            | RegraFechamentoBH        | getBancoHoras, getDataFinal, realizarFechamento                         |
| ContextoGeralRH          | qualquer regra           | getUsuarioAtivo, getUsuarioColaborador, getSaldoBancoHoras, getMesData  |

### Checklist pré-entrega (operacional)

- ☐ execute() sem throws Exception
- ☐ import com.senior.dataset.IEntity (não g5... )
- ☐ Sem .sc em R\*; .sc JSON com { ; id = filename
- ☐ MarcacaoRegra.getData().toLocalDate() ; MarcacaoAnterior só diferencaMinutos
- ☐ TipCon via SQL; IDs usuário long
- ☐ Filename = classe; auxiliares no nível da classe
- ☐ ICursor close no finally ; DBCenter session.close ; ContextSession **sem** close

### Itens pendentes / bloqueados até SDK (🚫)

| Item                                   | Status                                                                                                                                                                                |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Package/enum completo TipoHoraExtra    | 🚫 <b>Bloqueado até SDK</b> — usar só valores já citados no catálogo (TipoIntervalo etc.); demais → validacao_manual + perguntar enum do projeto. <b>Não</b> inventar como confirmada. |
| @Transactional vs container            | 🚫 <b>Bloqueado até SDK</b> — <b>não</b> anotar @Transactional sem evidência do projeto; deixar ao container.                                                                          |
| isNull/setNull em LocalDate de IEntity | Template IEntity = padrao_compilacao; se o SDK do projeto divergir → validacao_manual.                                                                                                |
| Capitalização getters USU_*            | <b>Norma fechada</b> (ver Acesso a dados): espelhar nome do campo — fora desta tabela.                                                                                                |

Sem jar/docs/projeto de referência, o teto de maturidade do treinamento fica ~**9.x**; nota **10** exige fechar os 🚫 com evidência de SDK.

Não inventar itens 🚫 como confirmada .

### Goldens ponta a ponta (sanitizados)

Referência obrigatória da Skill 1. Cada golden: LSP → inventário resumido → Java → checks Skill 5 esperados.

### G-SIT — HorSit / minutos

## LSP

```
Definir Numero nMin;  
nMin = HorSit[1];  
HorSit[1] = nMin + 60;
```

**Inventário:** HorSit[1] → getHorSit / setHorSit ; evidência confirmada ; contextoapuracao .

## Java

```
@Rule(description = "G-SIT sanitizado")  
public class RegraGSIT extends Apuracao {  
    @Override  
    public void execute() {  
        ContextoApuracao ctx = getContainer().getContextoApuracao();  
        int nMin = ctx.getHorSit(1);  
        ctx.setHorSit(1, nMin + 60);  
    }  
}
```

**Skill 5:** PASS CHK-SITAPI , CHK-MIN , CHK-THROWS , CHK-STUB . FAIL se  
getSituacao().setMinutos .

## G-CUR — cursor R\* (API / ContextSession, sem .sc )

## LSP

```
Definir Alfa aNomEmp;  
CriarCursor('R030EMP');  
AbrirCursor('R030EMP');  
// leitura NomEmp ...  
FecharCursor('R030EMP');
```

**Inventário:** cursor R030EMP nativo → não .sc ; preferir API/readonly; fallback ContextSession; evidência  
padrao\_compilacao .

## Java (padrão ContextSession se sem API semântica)

```
@Rule(description = "G-CUR sanitizado")  
public class RegraGCUR extends Apuracao {  
    @Override  
    public void execute() {  
        ContextoApuracao ctx = getContainer().getContextoApuracao();  
        String aNomEmp = "";  
        try {  
            IResultSet rs = ContextSession.getSession().executeQuery(  
                "SELECT NOMEMP FROM R030EMP WHERE NUMEMP = ?",  
                ctx.getColaborador().getNumEmp());  
            if (rs.next()) {  
                aNomEmp = rs.getString(0);  
            }  
        }  
    }  
}
```

```

    } catch (Exception ex) {
        ctx.mensagemLog("Erro G-CUR: " + ex.getMessage());
    }
    // ContextSession: não fechar sessão do container
}
}

```

**Skill 5:** PASS CHK-SCNAT (sem .sc em R\*), CHK-SQL2 , CHK-DBSESS , CHK-STUB . Preferir API readonly/ getDefinicao... quando existir no catálogo.

## G-USU — USU\_\* + I\* + .sc

### LSP

```

Definir Numero nCod;
CriarCursor('USU_TExemplo');
AbrirCursor('USU_TExemplo');
// lê USU_CodEmp ...
FecharCursor('USU_TExemplo');

```

**Inventário:** USU\_TExemplo custom → IUsuTExemplo + ScUsuTExemplo.sc + ICursor; evidência padrao\_compilacao .

### Java (classe)

```

@Rule(description = "G-USU sanitizado")
public class RegraGUSU extends Apuracao {
    @Override
    public void execute() {
        ContextoApuracao ctx = getContainer().getContextoApuracao();
        ICursor<IUsuTExemplo> cur =
getContainer().getEntitySession().newCursor(IUsuTExemplo.class);
        try {
            cur.addFilter("USU_CodEmp = :e",
                new MappedParamProvider("e", ctx.getColaborador().getNumEmp()));
            cur.open();
            if (cur.first()) {
                IUsuTExemplo row = cur.read();
                int nCod = row.isUSU_CodEmpNull() ? 0 : row.getUSU_CodEmp();
                ctx.mensagemLog("USU_CodEmp=" + nCod);
            }
        } finally {
            cur.close();
        }
    }
}

```

**Interface (trecho):** getUSU\_CodEmp / setUSU\_CodEmp espelhando o campo.

**.sc :** JSON começando com { ; "id": "ScUsuTExemplo" = filename sem extensão; entity = package+interface; **somente** USU\_\* .

**Skill 5:** PASS `CHK-FIN` , `CHK-SCJSON` , `CHK-SCID` , `CHK-SCNAT` , `CHK-STUB` .

## Saída para a Skill 1

```
contexto: Apuracao | FechamentoBH | outro | indefinido
item_lsp: ...
equivalente_java: ...
evidencia: confirmada | padrao_compilacao | padrao_anexo | validacao_manual
fonte: equivalencia-funcoes-regras | indice-funcoes | compilacao | anexo
limite: ...
```

## Relacionados

Skill 2 (URLs/aliases) · Skill 1 · Skill 5 ( `CHK-SITAPI` , `CHK-THROWS` , `CHK-SCNAT` , `CHK-TIPCON` , `CHK-MARANT` , ...)